Open in app ↗

Medium Search Write

Cybersecurity ✓ IOS App Development + Pentesting + Bug Bounty +

iOS SSL Pinning Bypass: Every Technique Ranked in 2026

A Akifkhan Follow 15 min read · Jun 1, 2026

52

Tags: `ios-security` `ssl-pinning` `mobile-pentesting` `frida` `objection` `burp-suite` `ethical-hacking` `cybersecurity` `bug-bounty` `infosec`

Estimated read time: 22 minutes

> *"The app doesn't trust your certificate. On iOS, it has four different ways to say so. Here is how you defeat each one."*

The Error That Is Different Every Time

You configured Burp Suite. You installed the CA certificate. You enabled full trust in Certificate Trust Settings — the step everyone misses the first time (Part 1 covered this).

You open the app. And you get one of these:

```
The certificate for this server is invalid.
```

Or the app loads but Burp shows nothing — silent failure, no error, just empty HTTP history while the app happily displays data it fetched from somewhere you cannot see.

Or Burp shows some requests but not others — partial interception, with certain API calls invisible.

You try Objection. You run `ios sslpinning disable`. Burp still shows nothing for those endpoints.

Here is what is happening: iOS has not one SSL pinning mechanism but several, implemented at different layers of the networking stack. Each layer requires a different bypass. Each bypass technique works on some implementations and fails on others.

This article covers every technique, ranked from easiest to most powerful. It is the iOS equivalent of Android Part 3 — same structure, different platform, different technical details that matter.

By the end you will have five bypass techniques and a decision tree that tells you exactly which one to try next when the current one fails.

iOS SSL Pinning: Four Implementation Layers

Before any bypass, the diagram. iOS apps can implement SSL pinning at four distinct layers — each requiring a different approach to defeat.

```
iOS NETWORK STACK — WHERE PINNING CAN LIVE:
──────────────────────────────────────────────────────────────

  App Code
     │
     ├──► Layer 1: NSAppTransportSecurity (Info.plist)
     │      Certificate authority restrictions, cleartext exceptions
     │      Bypass: edit Info.plist in IPA and repackage
     │
     ├──► Layer 2: NSURLSession delegate — didReceiveChallenge
     │      App manually validates the server certificate
     │      Bypass: hook the delegate method, return .useCredential
     │
     ├──► Layer 3: TrustKit / Alamofire / AFNetworking
     │      Third-party libraries with their own pinning logic
     │      Bypass: hook library-specific validation methods
     │
     └──► Layer 4: SecTrustEvaluate / SecTrustEvaluateWithError
           Apple's core Security framework validation
           Bypass: hook native C functions directly
──────────────────────────────────────────────────────────────
```

```
WHAT OBJECTION COVERS:     Layer 2, 3 (partially)
WHAT FRIDA COVERS:         Layer 2, 3, 4
WHAT IPA PATCHING COVERS:  Layer 1
WHAT NETWORK LAYER HOOKS:  Bypass all layers entirely
──────────────────────────────────────────────────────────────
```

The techniques below address each layer. Start at Technique 1. Work down until traffic appears in Burp.

Technique 1: Objection — One Command (Easiest)

Reliability: High for standard apps | Fails on: custom delegates, TrustKit, native pinning

The moment of confusion: You run `ios sslpinning disable` in Objection. It reports success. Burp still shows nothing for certain endpoints.

Objection's iOS SSL bypass hooks a specific list of common pinning delegate methods. If the app uses a delegate pattern or library not in Objection's hook list, it misses.

When it works: Apps using standard `URLSession` with delegate-based pinning, Alamofire's default session manager, or AFNetworking's standard pinning.

Start here. Always. If it works, you are done in 30 seconds.

```
# Ensure Frida server is running on jailbroken iPhone
# Connect iPhone via USB
```

```
# Spawn the app with Objection
objection --gadget com.target.app explore

# Inside the Objection REPL:
ios sslpinning disable
```

```
# Also run the comprehensive check:
ios sslpinning disable --quiet

# Additional Objection SSL commands:
ios certificates list    # list all pinned certificates
ios nsurlsession dump   # dump session configuration
```

Watch Burp Suite's HTTP History after running the command. Interact with the app — navigate screens that make network requests. If requests appear: success.

If Burp shows partial traffic (some requests visible, others not), the app uses multiple pinning implementations across different networking clients. Run Objection and then layer a Frida script on top.

The habit to build here: Run Objection first on every new target — even if you expect it to fail. The output tells you which methods Objection attempted to hook. Methods that were found but did not fix the issue narrow down which layer the pinning lives on, guiding your next step precisely.

Now make it harder: Objection hooks fire and report success, but traffic is still absent. The pinning is implemented in a TrustKit configuration loaded at app startup — before Objection's hooks are active. Move to Technique 2 with spawn mode to hook before startup code runs.

Technique 2: Frida Universal SSL Bypass Script

Reliability: Very high | Fails on: native-layer pinning, heavily custom implementations

The moment of confusion: Objection failed or partially worked. You need a wider net — something that hooks every known iOS SSL pinning implementation simultaneously.

The most maintained universal iOS SSL bypass script in 2025 is from iOS SSL Kill Switch, adapted for Frida. It hooks all four layers of the iOS network stack in one script.

```
// ios_ssl_bypass.js — Universal iOS SSL pinning bypass
// Hooks all major iOS SSL pinning implementations
```

```
// ── Layer 4: Core Security framework hooks ──────────────────────
// These are C functions — use Module.findExportByName

var SecTrustEvaluate = Module.findExportByName(
    "Security", "SecTrustEvaluate"
);
if (SecTrustEvaluate) {
    Interceptor.replace(SecTrustEvaluate, new NativeCallback(
        function(trust, result) {
            // kSecTrustResultProceed = 1
            if (!result.isNull()) {
                Memory.writeU32(result, 1);
            }
            return 0; // errSecSuccess
        },
        'int', ['pointer', 'pointer']
    ));
    console.log("[✓] SecTrustEvaluate hooked");
}

var SecTrustEvaluateWithError = Module.findExportByName(
    "Security", "SecTrustEvaluateWithError"
);
if (SecTrustEvaluateWithError) {
    Interceptor.replace(SecTrustEvaluateWithError, new
NativeCallback(
        function(trust, error) {
            if (!error.isNull()) {
                Memory.writePointer(error, NULL);
            }
            return 1; // true = trusted
```

```
        },
        'bool', ['pointer', 'pointer']
    ));
    console.log("[✓] SecTrustEvaluateWithError hooked");
}

// ── Layer 2 & 3: Objective-C delegate hooks ──────────────────
ObjC.schedule(ObjC.mainQueue, function() {

    // ── NSURLSession delegate pinning ────────────────────────
    // Hook URLSession:didReceiveChallenge:completionHandler:
    try {
        var NSURLSession = ObjC.classes.NSURLSession;
        var hookTarget =
"URLSession:didReceiveChallenge:completionHandler:";

        // Enumerate all classes implementing this delegate method
        var classList = ObjC.enumerateLoadedClassesSync();
        classList.forEach(function(cls) {
            try {
                var c = ObjC.classes[cls];
                if (c["- " + hookTarget] || c["- " + hookTarget]) {
                    Interceptor.attach(
                        c["- " + hookTarget].implementation ||
                        c["- " + hookTarget].implementation,
                        {
                            onEnter: function(args) {
                                // args[4] = completionHandler block
                                var handler = new
ObjC.Block(args[4]);
                                var origImpl =
handler.implementation;

                                handler.implementation = function(
                                    disposition, credential
                                ) {
                                    //
NSURLSessionAuthChallengeUseCredential = 0
                                    // Create credential from the
challenge
                                    var challenge = new
ObjC.Object(args[3]);
                                    var cred = ObjC.classes
                                        .NSURLCredential
                                        .credentialForTrust_(

challenge.protectionSpace()
                                                .serverTrust()
                                        );
                                    origImpl.call(this, 0,
cred.handle);
                                    console.log("[✓] NSURLSession
challenge bypassed");
```

```
                                    };
                                }
                            }
                        );
                        console.log("[✓] Hooked delegate: " + cls);
                    }
                } catch(e) {}
            });
        } catch(e) {
            console.log("[-] NSURLSession delegate hook error: " + e);
        }

        // ── TrustKit bypass ──────────────────────────────────────
        try {
            var TrustKit = ObjC.classes.TrustKit;
            if (TrustKit) {
                Interceptor.attach(
                    TrustKit["+ initSharedInstanceWithConfiguration:"]
                        .implementation,
                    {
                        onEnter: function(args) {
                            // Replace config with empty dict — disables
pinning
                            var NSMutableDictionary = ObjC.classes
                                .NSMutableDictionary;
                            args[2] =
NSMutableDictionary.dictionary().handle;
                            console.log("[✓] TrustKit config cleared");
                        }
                    }
                );
            }
        } catch(e) {}

        // ── Alamofire ServerTrustManager bypass ──────────────────
        try {
            var ServerTrustManager = ObjC.classes.ServerTrustManager;
            if (ServerTrustManager &&
                ServerTrustManager["- isValidHost:"]) {
                Interceptor.attach(
                    ServerTrustManager["- isValidHost:"].implementation,
                    {
                        onLeave: function(retval) {
                            retval.replace(1); // Always valid
                            console.log("[✓] Alamofire ServerTrustManager
bypassed");
                        }
                    }
                );
            }
        } catch(e) {}
```

```
    // ── AFNetworking bypass ──────────────────────────────────
    try {
        var AFSecurityPolicy = ObjC.classes.AFSecurityPolicy;
        if (AFSecurityPolicy) {
            Interceptor.attach(
                AFSecurityPolicy[
                    "- evaluateServerTrust:forDomain:"
                ].implementation,
                {
                    onLeave: function(retval) {
                        retval.replace(1);
                        console.log("[✓] AFNetworking
AFSecurityPolicy bypassed");
                    }
                }
            );
        }
    } catch(e) {}

    console.log("\n[*] All SSL bypass hooks active");
    console.log("[*] Interact with the app — traffic should appear in
Burp\n");
});
```

Run it with spawn mode — ensures hooks are in place before any startup pinning configuration loads:

```
frida -U -f com.target.app \
  -l ios_ssl_bypass.js \
  --no-pause
```

The habit to build here: After running the bypass, immediately open a screen that makes API calls and check Burp. Do not wait — some hooks only fire on the first request after app launch. If traffic still does not appear, add `console.log` calls inside each hook's `onEnter` to confirm which hooks are actually firing. Silent hooks that never trigger mean the pinning is at a different layer.

Now make it harder: The `SecTrustEvaluateWithError` hook fires (you see the console log) but Burp still shows nothing. The app has a secondary validation step in its custom networking layer. Move to Technique 3.

Technique 3: SSL Kill Switch 2 (Jailbreak — System-Wide)

Reliability: Very high | Requires: jailbroken device | Works on: almost everything except native custom TLS

The moment of confusion: Frida hooks fire, TrustKit configuration is cleared, AFNetworking is bypassed — and traffic is still absent. The pinning is implemented deeply enough that app-level hooks cannot reach it in your current script.

SSL Kill Switch 2 operates differently from every other technique. Instead of hooking app-level code, it patches Apple's `libboringssl` — the underlying TLS library that all iOS networking ultimately uses. It disables certificate validation at the system level, beneath every application framework.

Installation:

```
# On jailbroken iPhone — install via Cydia or Sileo
# Add source: https://julioverne.github.io
# Search: SSL Kill Switch 2
# Install
```

```
# Or install manually:
# Download .deb from github.com/nabla-c0d3/ssl-kill-switch2
```

```
dpkg -i ssl-kill-switch2.deb
killall -HUP SpringBoard   # restart SpringBoard to apply

# Enable in Settings → SSL Kill Switch 2 → toggle ON
# This applies globally to ALL apps until you toggle it off
```

Once enabled, relaunch the target app. All TLS validation is disabled system-wide. Burp intercepts everything.

Important: Disable SSL Kill Switch 2 when you are not actively testing. With it enabled, your test device is vulnerable to real MITM attacks — any network attacker can intercept all traffic. Toggle it on for a test session, off when done.

Verify it is working:

```
# On jailbroken device via SSH
# Check kill switch status
defaults read /var/mobile/Library/Preferences/\
  com.nablac0d3.SSLKillSwitch2.plist killSwitchEnabled
# Output: 1 (enabled) or 0 (disabled)
```

The habit to build here: Keep SSL Kill Switch 2 installed but disabled by default. When Frida-based approaches fail, enable it, test, then immediately disable it. Treat it as a circuit breaker — the most powerful tool, used when others fail, not as a default.

Now make it harder: SSL Kill Switch 2 is enabled but the app still does not connect to its API at all — not even failing gracefully, just showing a network error. The app detects that certificate validation is disabled and refuses to

run. This is a separate protection called **runtime integrity checking**. Move to Technique 4.

Technique 4: IPA Patching — Modify the App Directly

Reliability: Extremely high | Requires: no jailbreak | Fails on: app-level tamper detection

The moment of confusion: Every runtime hook has been defeated. Either the app detects Frida, detects SSL Kill Switch, or implements pinning at a layer none of the hooks reach.

The alternative: do not hook the running app. Modify the app itself before it runs.

IPA patching is iOS's equivalent of Android APK repackaging from Android Part 3. You extract the IPA, modify the configuration or inject a Frida gadget, repackage, re-sign, and reinstall.

Approach A: Patch Info.plist to Disable ATS

If the pinning is configured via NSAppTransportSecurity in Info.plist (Layer 1 from the diagram):

```
# Step 1: Extract the IPA
cp com.target.app.ipa com.target.app.zip
unzip com.target.app.zip -d ipa_extracted
```

```
# Step 2: Convert Info.plist to editable XML
plutil -convert xml1 \
  ipa_extracted/Payload/TargetApp.app/Info.plist \
  -o Info.plist.xml

# Step 3: Edit — replace NSAppTransportSecurity section
nano Info.plist.xml
```

Replace the entire `NSAppTransportSecurity` dictionary with:

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
    <key>NSAllowsArbitraryLoadsInWebContent</key>
    <true/>
</dict>
```

```
# Step 4: Convert back to binary plist
plutil -convert binary1 Info.plist.xml \
  -o ipa_extracted/Payload/TargetApp.app/Info.plist
```

```
# Step 5: Repackage
cd ipa_extracted
zip -r ../target_patched.ipa Payload/

# Step 6: Re-sign with your development certificate
# (Requires Apple Developer account — $99/year)
# Or use a free signing tool:

# Option A: AltStore + AltServer (free, works on Windows/Mac)
# Option B: Sideloadly (free)
# Option C: codesign directly on Mac:
codesign -f -s "iPhone Developer: Your Name" \
  ipa_extracted/Payload/TargetApp.app
```

Approach B: Inject Frida Gadget (No Jailbreak Required)

This approach works on any iOS device — no jailbreak needed. You inject Frida's dynamic library into the app binary, giving you a Frida server running inside the app's own process.

```
# Install objection's patchipa tool
pip3 install objection
```

```
# One command — patches the IPA with Frida gadget automatically
objection patchipa \
  --source com.target.app.ipa \
  --codesign-signature "iPhone Developer: Your Name (XXXXXXXXXX)"

# Output: com.target.app-frida-patched.ipa

# Install to device
ios-deploy --bundle com.target.app-frida-patched.ipa
# Or use ideviceinstaller:
ideviceinstaller -i com.target.app-frida-patched.ipa

# Connect Frida (the gadget is the server)
frida -U -n Gadget -l ios_ssl_bypass.js
```

For signing without a paid developer account — use `ios-app-signer` or AltStore:

```
# AltStore method (free — re-signs every 7 days)
# Install AltStore from altstore.io on your Mac
# Drag the patched IPA into AltStore → install
```

```
# The app installs and runs — Frida gadget starts automatically
# Connect from your computer:
frida -U Gadget
```

The habit to build here: Keep `objection patchipa` as a fallback tool ready to use. When all runtime bypass techniques fail, IPA patching with Frida gadget injection bypasses the entire problem — there is no runtime to detect Frida because Frida is already inside the app. The 20-minute patching process is worth it on targets where the runtime approach is completely blocked.

Now make it harder: The patched IPA will not install — the app uses a restrictive provisioning profile that blocks sideloading. This is common in enterprise apps distributed through MDM. In this case, you need either a developer account with explicit entitlements or a jailbroken device where MDM restrictions do not apply.

Technique 5: Hook at the Network Data Layer (Bypass Everything)

Reliability: Extremely high | Works on: all pinning implementations | Requires: Frida running

The moment of confusion: Every certificate-level bypass has failed. The app's pinning is implemented deep enough that intercepting at the TLS layer is not working. But the app is clearly making network requests — you can see the data appearing in the UI.

The solution: do not intercept at the TLS layer. Hook at the data layer — after the app has already decrypted the response, before it processes the plaintext data.

```
// hook_network_data_ios.js
// Bypasses SSL pinning entirely by reading data after decryption
```

```
ObjC.schedule(ObjC.mainQueue, function() {
    console.log("[*] Network data layer hook active");
    console.log("[*] SSL pinning irrelevant — reading post-
decryption\n");

    // ── Hook NSURLSession completionHandler
──────────────────────

    // This fires AFTER the response has been received and decrypted
    var NSURLSession = ObjC.classes.NSURLSession;

    Interceptor.attach(
        NSURLSession[
            "- dataTaskWithRequest:completionHandler:"
        ].implementation,
        {
            onEnter: function(args) {
                // Capture the original request details
                var request  = new ObjC.Object(args[2]);
                this.url     =
request.URL().absoluteString().toString();
                this.method  = request.HTTPMethod().toString();

                // Wrap the completion handler block
                var origBlock = new ObjC.Block(args[3]);
                var origImpl  = origBlock.implementation;
                var self_ref  = this;

                origBlock.implementation = function(data, response,
error) {

console.log("\n═══════════════════════════════════════════");
                    console.log("URL:    " + self_ref.url);
                    console.log("Method: " + self_ref.method);

                    // Read response data — already decrypted by the
OS
                    if (data && !new ObjC.Object(data).isNil()) {
                        var NSString = ObjC.classes.NSString;
                        var body = NSString.alloc()
                            .initWithData_encoding_(data, 4) // UTF-8
                            .toString();
                        if (body && body.length > 0) {
                            console.log("Body:   " +
body.substring(0, 2000));
                        }
                    }
```

```
                    if (response && !new
ObjC.Object(response).isNil()) {
                        var resp = new ObjC.Object(response);
                        try {
                            console.log("Status: " +
resp.statusCode());
                        } catch(e) {}
                    }

                    // Call original completion handler
                    origImpl.call(this, data, response, error);
                };
            }
        }
    );
    console.log("[✓] NSURLSession data hook active");

    // ── Hook NSJSONSerialization — catch all JSON parsing
──────────

    // Every JSON API response passes through this
    var NSJSONSerialization = ObjC.classes.NSJSONSerialization;

    Interceptor.attach(
        NSJSONSerialization[
            "+ JSONObjectWithData:options:error:"
        ].implementation,
        {
            onEnter: function(args) {
                var data = new ObjC.Object(args[2]);
                try {
                    var NSString = ObjC.classes.NSString;
                    var str = NSString.alloc()
                        .initWithData_encoding_(data, 4)
                        .toString();
                    if (str && str.length > 2) {
                        console.log("\n[JSON] Parsing: " +
                            str.substring(0, 1000));
                    }
                } catch(e) {}
            }
        }
    );
    console.log("[✓] NSJSONSerialization hook active");
});
```

This technique reads data after the OS has decrypted it — SSL pinning has no effect because you are not touching the TLS handshake at all. You are

reading the app's own variables after the network library has placed the decrypted response into them.

The habit to build here: When certificate-level bypasses are failing and the app is clearly receiving data, hook `NSJSONSerialization` first — it is a single hook that captures the plaintext content of every JSON API response in the entire app, regardless of which networking library fetched it or how the TLS was configured.

The Decision Tree: Which Technique to Use

```
App traffic not appearing in Burp
│
├─► Step 1: Try Objection
│     objection --gadget com.target.app explore
│     → ios sslpinning disable
│          │
│          ├─► Works? ──────────────────────────► DONE ✓
│          └─► Fails or partial?
│                    │
│                    ▼
├─► Step 2: Try Frida universal script
│     frida -U -f com.target.app -l ios_ssl_bypass.js
│          │
│          ├─► Works? ──────────────────────────► DONE ✓
│          └─► Hooks fire but no traffic?
│                    │
│                    ▼
├─► Step 3: SSL Kill Switch 2
│     Settings → SSL Kill Switch 2 → Enable
│          │
│          ├─► Works? ──────────────────────────► DONE ✓
│          └─► App refuses to connect with KS enabled?
│                    │
│                    ▼
├─► Step 4: IPA Patching
│     objection patchipa → re-sign → install
```

```
│     frida -U -n Gadget -l ios_ssl_bypass.js
│          │
│          ├─► Works? ──────────────────────────► DONE ✓
│          └─► Tamper detection / install blocked?
│                    │
│                    ▼
└─► Step 5: Hook at data layer
      Hook NSURLSession completionHandler
      Hook NSJSONSerialization
      Read post-decryption data directly          ► DONE ✓
```

Technique Comparison: iOS vs Android

Technique iOS Android Equivalent One-command bypass Objection `ios sslpinning disable` Objection `android sslpinning disable` Frida script Hook `SecTrustEvaluate` Hook `TrustManagerImpl.verifyChain` System-level bypass SSL Kill Switch 2 Magisk TrustUserCerts App patching `objection patchipa` APK repackaging + `network_security_config` Data layer hook `NSURLSession` completionHandler OkHttp `execute` hook

The methodology is identical to Android Part 3. The tool names and method names differ. The decision process is the same.

iOS-Specific Complications: What Makes This Harder Than Android

Code signing: Every iOS app binary must be signed with a valid certificate. Modifying the IPA breaks the signature — you must re-sign before reinstalling. Android APKs can be resigned with any key. iOS requires either a developer account or free tools that impose 7-day expiry limits.

No ADB equivalent: Android's ADB gives you a command-line interface to any device regardless of app. iOS requires SSH (jailbreak) or the device to be paired (non-jailbreak). The debugging surface is narrower.

App Store distribution: Apps from the App Store are encrypted (FairPlay DRM). The Mach-O binary inside the IPA is not directly readable until it is decrypted by a jailbroken device running tools like `frida-ios-dump` or `bagbak`. This is why getting the IPA requires either a jailbreak or alternative distribution sources for testing.

Entitlements matter: iOS entitlements (stored in the app binary's signature) control what the app is allowed to do. An app with `com.apple.developer.networking.networkextension` can create VPN tunnels that bypass your proxy entirely — and you cannot change entitlements without a matching provisioning profile.

Understanding these complications tells you why some bypasses require a jailbreak and others do not — and prevents you from spending hours on an approach that is architecturally impossible for a given target.

Practical Setup: Testing Without a Jailbreak

Many iOS security researchers test production apps without a jailbreak by combining three approaches:

Traffic interception: Burp Suite + iOS CA certificate trust (from Part 1). Works for apps without pinning or with ATS-only pinning.

IPA gadget injection: `objection patchipa` + AltStore sideloading. Works for apps without tamper detection. No jailbreak required.

Static analysis only: Extract IPA, read Info.plist, search binary strings, analyse JS bundles. Finds hardcoded secrets, ATS misconfigurations, and insecure configurations without any dynamic testing.

These three approaches together cover the majority of findings reported in iOS bug bounty programs — because most findings are either static analysis catches (hardcoded keys, ATS misconfig) or API-level issues (IDOR, broken auth) that do not require bypassing pinning at all.

Jailbreak-only techniques — Keychain dumping, filesystem access, deep Frida hooks — are needed for a small but critical category of findings. They are worth pursuing once you have the hardware. But do not let the absence of a jailbreak stop you from testing iOS apps.

Quick Reference: iOS SSL Bypass Commands

```
# ── Objection ──────────────────────────────────────────
objection --gadget com.target.app explore
# Inside REPL:
ios sslpinning disable
ios certificates list
```

```
# ── Frida script ───────────────────────────────────────
frida -U -f com.target.app -l ios_ssl_bypass.js --no-pause

# ── SSL Kill Switch 2 (jailbreak) ──────────────────────
# Install via Cydia/Sileo → enable in Settings
```

```
# Verify: defaults read /var/mobile/Library/Preferences/\
#   com.nablac0d3.SSLKillSwitch2.plist killSwitchEnabled

# ── IPA patching ──────────────────────────────────────────
objection patchipa \
  --source app.ipa \
  --codesign-signature "iPhone Developer: Name (ID)"
# Install → frida -U -n Gadget -l script.js

# ── Data layer hook ───────────────────────────────────────
frida -U -f com.target.app -l hook_network_data_ios.js --no-pause

# ── Info.plist ATS patch ──────────────────────────────────
plutil -convert xml1 Info.plist -o Info.xml
# Edit: add NSAllowsArbitraryLoads = true
plutil -convert binary1 Info.xml -o Info.plist
# Repackage → re-sign → install
```

One Strong Recommendation

Once your bypass is working and Burp is capturing iOS traffic, the next step is making Burp more powerful for iOS-specific testing. Install the **Burp Suite Mobile Assistant** from the BApp Store inside Burp.

Mobile Assistant provides iOS-specific Burp extensions that understand common iOS API patterns — JWT token extraction, iOS-specific header analysis, and automatic parameter discovery for SwiftUI-based apps. It is maintained by PortSwigger specifically for mobile testing and integrates directly with your existing Burp project.

Install it from Burp Suite → Extender → BApp Store → search “Mobile Assistant”. It is free and takes three minutes to install. Once you have iOS traffic flowing through Burp, Mobile Assistant makes the difference between manually reviewing hundreds of requests and having the most interesting ones automatically surfaced.

Your iOS Series — Progress Through Part 3

Part Topic Core skill 1 ✅ iOS Pentesting Intro Lab, IPA analysis, Burp setup, Info.plist 2 ✅ Frida on iOS Deep Dive ObjC hooking, Swift interop, runtime analysis 3 ✅ (this) iOS SSL Pinning Bypass Every technique ranked, decision tree 4 → iOS Keychain & Data Storage Finding secrets at rest 5 → iOS API Security Testing Same methodology, Apple ecosystem 6 → Your First iOS Bug Bounty Report From finding to payout

The next article covers the iOS data storage layer — Keychain accessibility levels, NSUserDefaults pitfalls, Data Protection classes, and how to find sensitive data at rest using both static and dynamic techniques. This is where the most consistently reportable iOS findings live.

Which technique finally got traffic into Burp? Drop the number in the comments. The distribution across techniques tells me which iOS pinning implementations are most common in the wild right now.

Follow for Part 4: iOS Keychain and Data Storage — Finding Secrets at Rest.

All techniques in this article are for educational purposes only. Test exclusively on devices you own or have explicit written authorization to test.

SEO Meta Description: Complete guide to iOS SSL pinning bypass in 2025 — covering Objection, Frida universal scripts hooking SecTrustEvaluate and NSURLSession delegates, SSL Kill Switch 2, IPA patching with Frida gadget injection, and network data layer hooking. Every technique ranked with a decision tree. Part 3 of the iOS pentesting series.

Focus Keywords: ios ssl pinning bypass 2025, bypass certificate pinning ios, frida ios ssl bypass, objection ios sslpinning disable, ssl kill switch 2 ios, ios ipa patching frida gadget, SecTrustEvaluate hook frida, ios mobile pentesting burp suite, ios certificate bypass tutorial

Cybersecurity ✓ | IOS App Development + | Pentesting + | Bug Bounty +



Written by Akifkhan

13 followers · 1 following

Follow